

```
-- =====
-- NexShop Group S.A. – Base de Datos
-- Archivo: consultas.sql
-- Descripcion: 14 consultas con comentario explicativo
-- =====
```

```
USE nexshop;
```

```
-- =====
-- CONSULTA 1 – SELECT *
-- Devuelve todos los empleados registrados en el sistema.
-- Util para el area de RRHH como listado completo de plantilla.
-- =====
```

```
SELECT * FROM empleado;
```

```
-- =====
-- CONSULTA 2 – Campos concretos
-- Devuelve solo nombre, apellidos y email de los clientes
-- registrados. Util para campañas de email marketing.
-- =====
```

```
SELECT nombre, apellidos, email
FROM cliente
WHERE activo = TRUE;
```

```
-- =====
-- CONSULTA 3 – WHERE con valor exacto
-- Muestra todos los pedidos online que estan en estado
```

```
-- 'pendiente'. Util para que logistica priorice su gestion.
-- =====
SELECT
    po.id_pedido,
    CONCAT(c.nombre, ' ', c.apellidos) AS cliente,
    po.fecha_pedido,
    po.total
FROM pedido_online po
JOIN cliente c ON po.id_cliente = c.id_cliente
WHERE po.estado = 'pendiente'
ORDER BY po.fecha_pedido ASC;
```

```
-- =====
-- CONSULTA 4 – LIKE con patron en campo de texto + JOIN
-- Busca productos cuyo nombre contenga la palabra 'portátil'.
-- Incluye la subcategoria a la que pertenecen.
-- Util para el buscador de la web (nexshop.es).
-- =====
SELECT
    p.id_producto,
    p.nombre AS producto,
    p.pvp,
    s.nombre AS subcategoria,
    c.nombre AS categoria
FROM producto p
JOIN subcategoria s ON p.id_subcategoria = s.id_subcategoria
JOIN categoria c ON s.id_categoria = c.id_categoria
WHERE p.nombre LIKE '%ortátil%'
```

```
    AND p.activo = TRUE
ORDER BY p.pvp ASC;
```

```
-- =====
-- CONSULTA 5 – LIKE que empieza por letra + LEFT JOIN
-- Lista los clientes cuyo nombre empieza por 'A', junto con
-- el numero de pedidos que han realizado (puede ser 0).
-- Util para filtrar clientes en atencion al cliente.
-- =====
```

```
SELECT
    c.id_cliente,
    CONCAT(c.nombre, ' ', c.apellidos) AS cliente,
    c.email,
    COUNT(po.id_pedido) AS total_pedidos
FROM cliente c
LEFT JOIN pedido_online po ON c.id_cliente = po.id_cliente
WHERE c.nombre LIKE 'A%'
GROUP BY c.id_cliente
ORDER BY total_pedidos DESC;
```

```
-- =====
-- CONSULTA 6 – BETWEEN fechas + JOIN
-- Pedidos online realizados en el primer trimestre de 2024,
-- con nombre del cliente y direccion de entrega.
-- Util para informes de ventas trimestrales.
-- =====
```

```
SELECT
```

```

    po.id_pedido,
    CONCAT(c.nombre, ' ', c.apellidos)          AS cliente,
    po.fecha_pedido,
    po.estado,
    po.total,
    CONCAT(dc.calle, ' ', dc.numero, ' ', ' ',
           dc.ciudad)                          AS
direccion_entrega
FROM pedido_online po
JOIN cliente c          ON po.id_cliente      =
c.id_cliente

JOIN direccion_cliente dc ON po.id_direccion_entrega =
dc.id_direccion
WHERE po.fecha_pedido BETWEEN '2024-01-01 00:00:00'
                                AND '2024-03-31 23:59:59'

ORDER BY po.fecha_pedido ASC;

```

```

-- =====
-- CONSULTA 7 – BETWEEN numerico + agregacion
-- Precio medio de los productos cuyo PVP esta entre 100
-- y 800 euros, agrupado por subcategoria.
-- Util para el analisis de precios del departamento de compras.
-- =====

```

```

SELECT
    s.nombre          AS subcategoria,
    COUNT(p.id_producto) AS num_productos,
    ROUND(AVG(p.pvp), 2) AS precio_medio,
    MIN(p.pvp)          AS precio_minimo,
    MAX(p.pvp)          AS precio_maximo

```

```

FROM producto p
JOIN subcategoria s ON p.id_subcategoria = s.id_subcategoria
WHERE p.pvp BETWEEN 100 AND 800
      AND p.activo = TRUE
GROUP BY s.id_subcategoria
ORDER BY precio_medio DESC;

```

```

-- =====
-- CONSULTA 8 – Condicion numerica + LEFT JOIN
-- Lineas de pedido con cantidad superior a 1 unidad,
-- mostrando el nombre del producto y el pedido al que
-- pertenecen.
-- Util para detectar compras al por mayor o errores de entrada.
-- =====

SELECT
    lp.id_linea_pedido,
    lp.id_pedido,
    p.nombre           AS producto,
    lp.cantidad,
    lp.precio_unitario,
    ROUND(lp.cantidad * lp.precio_unitario
          * (1 - lp.descuento_aplicado / 100), 2) AS subtotal
FROM linea_pedido lp
LEFT JOIN producto p ON lp.id_producto = p.id_producto
WHERE lp.cantidad > 1
ORDER BY lp.cantidad DESC;

```

```

-- =====
-- CONSULTA 9 – Subconsulta: clientes con gasto superior
--          a la media + ORDER BY ASC
-- Clientes cuyo gasto total en pedidos online supera la media
-- del conjunto de clientes. Ordenados del menor al mayor gasto.
-- Util para el programa de fidelizacion (detectar clientes
VIP).
-- =====

SELECT
    CONCAT(c.nombre, ' ', c.apellidos) AS cliente,
    c.email,
    ROUND(SUM(po.total), 2)           AS gasto_total
FROM cliente c
JOIN pedido_online po ON c.id_cliente = po.id_cliente
WHERE po.estado <> 'cancelado'
GROUP BY c.id_cliente
HAVING gasto_total > (
    SELECT AVG(total_por_cliente)
    FROM (
        SELECT id_cliente, SUM(total) AS total_por_cliente
        FROM pedido_online
        WHERE estado <> 'cancelado'
        GROUP BY id_cliente
    ) AS sub
)
ORDER BY gasto_total ASC;

-- =====

```

```

-- CONSULTA 10 – ORDER BY DESC + subconsulta
-- Productos que tienen alguna promocion activa a fecha de hoy,
-- mostrando el PVP original y el precio con descuento.
-- Ordenados de mayor a menor descuento porcentual.
-- Util para el banner de ofertas de la web.
-- =====

SELECT
    p.nombre
AS producto,
    p.pvp
AS pvp_original,
    pr.descuento_porcentaje,
    ROUND(p.pvp * (1 - pr.descuento_porcentaje / 100), 2) AS
precio_con_descuento,
    pr.fecha_inicio,
    pr.fecha_fin
FROM producto p
JOIN promocion pr ON p.id_producto = pr.id_producto
WHERE CURDATE() BETWEEN pr.fecha_inicio AND pr.fecha_fin
ORDER BY pr.descuento_porcentaje DESC;

-- =====

-- CONSULTA 11 – Vista + saldo de puntos + ORDER BY alfabetico
-- Crea una vista con el saldo actual de puntos de cada cliente,
-- calculado siempre desde el historico de movimientos.
-- Se muestra ordenado alfabeticamente por apellido.
-- =====

CREATE OR REPLACE VIEW vista_saldo_puntos AS
SELECT

```

```

    c.id_cliente,
    CONCAT(c.nombre, ' ', c.apellidos) AS cliente,
    c.email,
    COALESCE(SUM(CASE WHEN mp.tipo = 'ganado' THEN mp.puntos
ELSE 0 END), 0) AS puntos_ganados,
    COALESCE(SUM(CASE WHEN mp.tipo = 'canjeado' THEN mp.puntos
ELSE 0 END), 0) AS puntos_canjeados,
    COALESCE(SUM(CASE WHEN mp.tipo = 'ganado' THEN mp.puntos
ELSE 0 END), 0)
    - COALESCE(SUM(CASE WHEN mp.tipo = 'canjeado' THEN mp.puntos
ELSE 0 END), 0) AS saldo_actual
FROM cliente c
LEFT JOIN movimiento_puntos mp ON c.id_cliente = mp.id_cliente
GROUP BY c.id_cliente;

```

-- Consulta sobre la vista, ordenada alfabeticamente:

```

SELECT *
FROM vista_saldo_puntos
ORDER BY cliente ASC;

```

-- =====

-- CONSULTA 12 – UPDATE campo concreto

-- Cambia el estado del pedido #7 a 'enviado' una vez

-- que logistica confirma la salida del almacen.

-- =====

```

UPDATE pedido_online
SET estado = 'enviado'
WHERE id_pedido = 7;

```


-- Verificacion del cambio:

```
SELECT id_pedido, estado, fecha_pedido, total
FROM pedido_online
WHERE id_pedido = 7;
```

-- =====

-- CONSULTA 13 – UPDATE con WHERE para identificar el registro

-- Actualiza el email corporativo del empleado con id 3

-- tras un cambio de nombre en la empresa.

-- =====

```
UPDATE empleado
```

```
SET email_corporativo = 'juan.martinez.updated@nexshop.es'
```

```
WHERE id_empleado = 3;
```

-- Verificacion del cambio:

```
SELECT id_empleado, nombre, apellidos, email_corporativo
FROM empleado
WHERE id_empleado = 3;
```

-- =====

-- CONSULTA 14 – JOIN multiple con logica de negocio

-- Muestra cada pedido online con el nombre completo del

-- cliente, su ciudad de entrega, numero de productos distintos

-- en el pedido y el total. Incluye descuento por puntos si aplica.

-- Util como resumen ejecutivo de pedidos para operaciones.

-- =====

SELECT

```
    po.id_pedido,
    CONCAT(c.nombre, ' ', c.apellidos)    AS cliente,
    c.email,
    dc.ciudad                            AS ciudad_entrega,
    po.fecha_pedido,
    po.estado,
    COUNT(lp.id_linea_pedido)            AS num_lineas,
    SUM(lp.cantidad)                     AS unidades_totales,
    po.descuento_puntos,
    po.total
```

FROM pedido_online po

```
JOIN cliente c          ON po.id_cliente      =
c.id_cliente
```

```
JOIN direccion_cliente dc ON po.id_direccion_entrega =
dc.id_direccion
```

```
JOIN linea_pedido lp     ON po.id_pedido      =
lp.id_pedido
```

GROUP BY po.id_pedido

ORDER BY po.fecha_pedido DESC;